

- them. A set of vertices, S , such that no two vertices in S have an edge between them, is known as an 'independent set' of the graph G . Given a graph $G(V, E)$, can you find the maximum independent set of G ? The maximum independent set is that independent set containing the maximum number of vertices among all independent sets. Given an arbitrary graph G , can you design a polynomial-time algorithm to find the maximum independent set of G ? If so, give the algorithm. If not, explain why no polynomial-time algorithm is possible? Would your answer change if the graph was determined to be a tree? If you are asked to colour a graph, and you are given the different independent sets present in the graph, how would you use that information to colour the graph with a minimum number of colours?
- Given strings A , B and C , find out if string C is an interleaved version of strings A and B . Basically, the same order of letters, but mixed together, is an interleaving. For example, $A = abcd$; $B = xyz$; $C = abxcydz$ is an interleaving—but not if C is $baxcdyz$. Write an algorithm to determine if string C is an interleaving of strings A and B . How would you handle the case if a character in A is also present in B ? What is the complexity of your algorithm?
 - when there are no duplicate characters in strings A and B , and
 - when there are duplicate characters in strings A and B ?
 - We all know that eggs break when dropped from a height. If you are present in a skyscraper with N floors, you are told that if you drop an egg from floor K or any floor above it, to the ground, then the egg will break. If you drop the egg from a floor below K to the ground, then it will not break. You are given a supply of eggs, and asked to determine the value of K . What would be your strategy of determining the value of K if you are given an infinite supply of eggs? How would your solution change if you are given only three eggs?
 - You are given two sets of integers—Set A and Set B , both containing n integers each. You are given a value ' k '. You are asked to find out whether there exists a pair of integers (a, b) such that $a \in A$, $b \in B$ and $a + b = k$. A brute-force method would be to test each pair of integers (a, b) . However, this would require a complexity of $O(n^2)$. Can you design an algorithm which can be more efficient than the brute-force algorithm? Can you design a solution which has $O(n \log n)$ complexity?
 - We are familiar with different synchronisation mechanisms like mutex, semaphore, etc. Is it possible to implement these different synchronisation mechanisms if the underlying architecture does not support an atomic way of testing a memory location for the presence of a specific value, and changing it to a new value? (By

- atomic, I mean that both the events—the testing of the memory location and the writing of a new value to it—either happen successfully, or neither of them happens.)
- We are all familiar with algorithms for finding the median in linear time. A popular one is the variant of quick-sort based on the partition procedure of quick-sort, so that we limit our search for the median continuously to one side of the given array. (For a detailed discussion of the implementation, see http://en.wikipedia.org/wiki/Selection_algorithm) However, consider a large stream of integers that are continuously getting generated and passed on to you. You are asked to return the median for the stream of integers already received. (*Clue:* Remember that you are asked for the median, multiple times. So spending a lot of effort on setting up a data structure such that repeated queries are faster, would amortise over the multiple queries you would receive.) Design an algorithm that returns the median for the stream of integers that have been received till that point in time. What is the complexity of your algorithm?

My 'must-read book' for this month

This month's suggestion for the must-read book/article comes from me, though it actually is neither a single book, nor an article. I wanted to suggest that you look through the online algorithms course on the COURSERA website (<http://www.coursera.org/courses>). The algorithms course is taught by Robert Sedgewick, author of the popular book on data structures.

Another online course I would like to recommend to our readers is the 'Design of Computer Programs (CS212) Programming Principles', which is available at Udacity (<http://www.udacity.com/>). This is an excellent course to improve your problem solving skills.

If you have a favourite programming book or article that you think is a must-read for every programmer, please do send me a note with the book's name, and a short write-up on why you think it is useful, so I can mention it in this column. It would help many readers who want to improve their coding skills.

If you have any favourite programming puzzles that you would like to discuss on this forum, please send them to me, along with your solutions and feedback, at sandyasm_AT_yahoo_DOT_com. Till we meet again next month, happy programming and here's wishing you the very best! 

By: Sandya Mannarswamy

The author is an expert in systems software, and is currently working as a researcher in IBM India Research Labs. Her interests include compilers, multi-core technologies and software development tools. If you are preparing for systems software interviews, you may find it useful to visit Sandya's LinkedIn group, 'Computer Science Interview Training India' at <http://www.linkedin.com/groups?home=&gid=2339182>.